HireX API Endpoint Integration and Concepts Guide

Purpose

This document explains how the HireX frontend connects to the FastAPI backend, how many endpoints exist, what each endpoint does, where the request starts, where it ends, and which technical concepts are used in the process. This is made for lead/mentor discussion, so the focus is on API flow, frontend-backend connection, async calls, dynamic loading, and the vocabulary around the project.

Endpoint Count

HireX currently has **41 backend endpoints**:

- **40 `/api/*` product endpoints**
- **1 `/health` system endpoint**

Endpoint group count:

Area	Prefix	Count	Purpose
Auth	`/api/auth`	4	Login, current user, registration, hidden manager bootstrap
Candidate profile	`/api/candidates`	2	Candidate profile read/update
Resumes	`/api/resumes`	2	Resume upload/list
Jobs	`/api/jobs`	5	Job CRUD
Applications	`/api/applications`	2	Apply to job and list applications
Admin dashboard	`/api/admin`	7	HR dashboard, candidate evidence, decisions, analytics
Validator	`/api/validator`	5	Direct validation, bulk validation, result lookup, R1 queue
Excel intake	`/api/excel-intake`	8	JD configuration, validator batch run, batch management
Talent pool	`/api/talent-pool`	5	Reusable candidate pool, batch memberships, stage decisions
Health	`/health`	1	Backend health check

High-Level API Architecture

The frontend is a Next.js application. The backend is a FastAPI application. The connection is done through a shared Axios client:

```
Frontend screen
  -> apiClient request
  -> FastAPI router endpoint
  -> auth/session dependency
  -> service layer
  -> repository / SQLAlchemy query
  -> PostgreSQL database
  -> Pydantic response
  -> React state update
  -> UI re-render
```

The shared API client is in:

```
frontend/src/api/client.ts
```

It sets the backend base URL:

```
http://127.0.0.1:8000/api
```

It also attaches the JWT token from `localStorage` to every request:

```
Authorization: Bearer <hirex_token>
```

So when the frontend calls:

```
apiClient.get("/admin/candidates")
```

the actual backend URL becomes:

```
GET http://127.0.0.1:8000/api/admin/candidates
```

Router Registration Start Point

Backend API routing starts in:

```
hirex/main.py
```

That file creates the FastAPI app and attaches routers:

Router	Prefix	Connected file
Auth	`/api/auth`	`hirex/api/auth/routes.py`
Candidates	`/api/candidates`	`hirex/api/candidate/routes.py`
Resumes	`/api/resumes`	`hirex/api/resumes/routes.py`
Jobs	`/api/jobs`	`hirex/api/jobs/routes.py`
Applications	`/api/applications`	`hirex/api/applications/routes.py`
Admin	`/api/admin`	`hirex/api/admin/routes.py`
Validator	`/api/validator`	`hirex/api/validator/routes.py`

Router	Prefix	Connected file
Excel intake	`/api/excel-intake`	`hirex/api/excel_intake/routes.py`
Talent pool	`/api/talent-pool`	`hirex/api/talent_pool/routes.py`

This means `main.py` is the backend API entry point. The actual business work is not done there. It only connects routers.

Frontend API Start Points

The main frontend screens that call APIs are:

Frontend file	Main API usage
`frontend/src/screens/auth/LoginPage.tsx`	Login and current user check
`frontend/src/screens/admin/OperationsDashboardPage.tsx`	JD config, run validator, candidate review, HR decision
`frontend/src/screens/admin/BatchesPage.tsx`	Batch history and batch membership view
`frontend/src/screens/jobs/JobsPage.tsx`	Job listing and apply
`frontend/src/screens/applications/ApplicationsPage.tsx`	Application listing
`frontend/src/screens/candidate/ProfilePage.tsx`	Candidate profile load/update

Most Important Flow: HR Runs Validator

This is the core flow of the current project.

```
HR clicks Run Validator
  -> OperationsDashboardPage.runIntake()
  -> POST /api/talent-pool/screen if reusable pool exists
  -> POST /api/excel-intake/run if more candidates are needed
  -> ExcelIntakeService.run()
  -> reads JD workbook
  -> reads candidate workbook
  -> CandidateIdentityService deduplicates candidates
  -> ValidatorService evaluates applications
  -> ValidatorResult rows are saved
  -> CandidateBatchMembership rows are saved
  -> shortlisted/rejected Excel files are exported
  -> dashboard reloads selected batch
```

Important files:

Step	File
Button click and API call	`frontend/src/screens/admin/OperationsDashboardPage.tsx`
Endpoint	`hirex/api/excel_intake/routes.py`
Main orchestration	`hirex/excel_intake/service.py`
Candidate dedupe	`hirex/services/candidate_identity_service.py`
Validator scoring	`hirex/validator/service.py`
Batch membership	`hirex/services/batch_tracking_service.py`
Dashboard read model	`hirex/services/admin_dashboard_service.py`

API Endpoint Map

Auth APIs

Method	Endpoint	Frontend usage	Backend handler	What it does
POST	`/api/auth/register`	Candidate registration page	`register()`	Creates a candidate user account
POST	`/api/auth/login`	Login page	`login()`	Verifies email/password and returns JWT token
GET	`/api/auth/me`	Login redirect check	`me()`	Returns current logged-in user from token
POST	`/api/auth/bootstrap-manager`	Script/admin setup	`bootstrap_manager()`	Creates recruiter/hr/admin using bootstrap key; hidden from schema

Lead question: How is login connected?

Answer: Login starts in `LoginPage.tsx`, calls `POST /auth/login`, backend `AuthService.login()` validates credentials, returns a token, frontend stores it in `localStorage`, then Axios interceptor sends it on future API calls.

Admin Dashboard APIs

Method	Endpoint	Frontend usage	Backend handler	What it does
GET	`/api/admin/summary`	Dashboard metric cards	`summary()`	Returns total/pass/review/f counts, optionally
GET	`/api/admin/pool-analytics`	Pool analytics widgets	`pool_analytics()`	Returns source m reason counts, po
GET	`/api/admin/reason-bank`	HR decision modal	`reason_bank()`	Returns standard and labels
GET	`/api/admin/sourcing-batches`	Future batch audit	`sourcing_batches()`	Returns sourcing/ metadata
GET	`/api/admin/candidates`	Candidate table	`candidates()`	Returns paginated rows with filters
GET	`/api/admin/candidates/{application_id}`	Candidate detail drawer	`candidate_detail()`	Returns resume s evidence, timeline changes
POST	`/api/admin/candidates/{application_id}/decision`	HR review action	`decide_candidate()`	Records HR actio candidates

Admin API connection:

```
OperationsDashboardPage.loadDashboard()
  -> GET /admin/summary
  -> GET /admin/candidates
  -> AdminDashboardService
```

```
   -> SQLAlchemy joins ValidatorResult, Application, CandidateProfile, Job, HRReviewAction
   -> dashboard cards and candidate table re-render
```

Candidate detail connection:

```
Open candidate button
  -> openCandidate()
  -> GET /admin/candidates/{application_id}?validator_result_id=...
  -> AdminDashboardService.detail()
  -> joins parsed resume, validator scores, timeline, refresh changes
  -> detail drawer renders evidence
```

HR decision connection:

```
HR chooses Move Forward / Hold / Reject
  -> submitAction()
  -> POST /admin/candidates/{application_id}/decision
  -> AdminDashboardService.review()
  -> HRReviewAction row is inserted
  -> CandidateBatchMembership stage is updated
  -> dashboard reloads candidate list
```

Excel Intake APIs

Method	Endpoint	Frontend usage	Backend handler
GET	`/api/excel-intake/configuration`	Page load	`configuration()`
POST	`/api/excel-intake/requirements`	Save JD	`save_requirement()`
PATCH	`/api/excel-intake/requirements/{requirement_id}/status`	Active/draft/inactive toggle	`set_requirement_status()`
POST	`/api/excel-intake/run`	Run validator	`run_excel_intake()`
GET	`/api/excel-intake/batches/latest`	Interrupted request recovery	`get_latest_excel_intake_batch()`
GET	`/api/excel-intake/batches`	Batch dropdown and batch screen	`list_excel_intake_batches()`
GET	`/api/excel-intake/batches/{batch_id}`	Poll/recovery	`get_excel_intake_batch()`
DELETE	`/api/excel-intake/batches/{batch_id}`	Delete batch	`delete_excel_intake_batch()`

Validator run connection:

```
runIntake()
  -> POST /excel-intake/run
  -> ExcelIntakeService.run()
  -> read_requirement()
  -> read_candidates()
  -> _upsert_job()
  -> _upsert_candidate_application()
  -> ValidatorService.evaluate_application()
  -> track_validator_result()
  -> export_results()
  -> ExcelIntakeResult returned
```

Why this API is important: this is the current MVP replacement for candidate portal sourcing. HR can keep Excel as input, but the core system still stores data in Postgres and shows results in the dashboard.

Talent Pool APIs

Method	Endpoint	Frontend usage	Backend handler	What it does
GET	`/api/talent-pool/summary`	Dashboard pool cards	`summary()`	Returns available/reusable/stale/in-process counts
POST	`/api/talent-pool/screen`	Pool-first validation	`screen()`	Revalidates reusable candidates before new sourcing
POST	`/api/talent-pool/candidates/{candidate_id}/release`	Future manual release	`release()`	Releases candidate back to pool with reason
POST	`/api/talent-pool/memberships/{membership_id}/stage-decision`	Future R1/T1/T2 stage moves	`stage_decision()`	Records stage decision and updates workflow status
GET	`/api/talent-pool/batches/{batch_id}/memberships`	Batches page	`batch_memberships()`	Lists candidate membership for one batch

Reusable pool connection:

```
runIntake()
  -> checks talentPool.screen_ready
  -> POST /talent-pool/screen
  -> TalentPoolService.screen()
  -> validates stored reusable candidates first
  -> if not enough candidates, POST /excel-intake/run fetches new candidates
```

This solves the lead's concern that rejected candidates should not be lost. They are stored once and reused in future openings.

Validator APIs

Method	Endpoint	Frontend usage	Backend handler	Wh do
GET	`/api/validator/queues/r1`	Future agent/HR queue	`list_r1_queue()`	Lis ca rou R1

Method	Endpoint	Frontend usage	Backend handler	Wh do
POST	`/api/validator/evaluate`	Direct validation API	`evaluate_application()`	Ev on ap
GET	`/api/validator/result/{validator_result_id}`	Audit lookup	`get_validator_result()`	Ge va re
GET	`/api/validator/application/{application_id}`	Application lookup	`get_application_validator_result()`	Ge re ap
POST	`/api/validator/bulk-evaluate`	Bulk validation API	`bulk_evaluate()`	Ev se ap

Important note: In the current dashboard flow, the frontend usually does not call `/api/validator/evaluate` directly. Excel intake calls `ValidatorService` internally. The validator API still exists for direct application validation and future agent workflows.

Jobs, Applications, Candidate Profile, Resume APIs

Method	Endpoint	Frontend usage	Backend handler	What it does
GET	`/api/jobs/list`	Jobs page	`list_jobs()`	Shows open jobs
POST	`/api/jobs/create`	Admin/future job create	`create_job()`	Creates a job
GET	`/api/jobs/{job_id}`	Job detail	`get_job()`	Gets one job
PUT	`/api/jobs/{job_id}`	Job update	`update_job()`	Updates one job
DELETE	`/api/jobs/{job_id}`	Job delete	`delete_job()`	Deletes one job
POST	`/api/applications/apply`	Jobs page apply button	`apply_to_job()`	Creates job application
GET	`/api/applications/list`	Applications page	`list_applications()`	Lists candidate applications
GET	`/api/candidates/profile`	Profile page	`get_profile()`	Loads current user's profile
PUT	`/api/candidates/profile/update`	Profile page save	`update_profile()`	Updates profile fields
POST	`/api/resumes/upload`	Resume upload	`upload_resume()`	Uploads resume file
GET	`/api/resumes/list`	Resume list	`list_resumes()`	Lists uploaded resumes

These are foundation APIs from the candidate portal approach. They are still useful because the project can later switch from Excel-first to candidate-portal-first without rebuilding the validator core.

Frontend Dynamic Loading Concepts Used

`useEffect`

`useEffect` runs side effects after React renders. In HireX, we use it to load data from APIs when a page opens or when selected filters change.

Example usage:

```
OperationsDashboardPage opens
  -> useEffect runs
  -> loadConfiguration()
  -> loadBatches()
  -> load talent pool summary
  -> load analytics and reason bank
```

Another example:

```
selected batch changes
  -> useEffect detects dependency change
  -> API reloads batch memberships
  -> BatchesPage re-renders selected batch details
```

`useCallback`

`useCallback` keeps a function stable between renders unless its dependencies change. We use it for API loader functions like `loadDashboard`, `loadConfiguration`, and `loadBatches`.

Why we use it: because these functions are dependencies of `useEffect`. Without `useCallback`, they could be recreated too often and cause extra API calls.

`useState`

`useState` stores UI state: selected JD, selected batch, candidates, loading state, error message, selected candidate, modal status, HR reason codes, page number.

Example:

```
setCandidates(response.data.items)
  -> React state changes
  -> candidate table re-renders
```

Dynamic Rendering

Dynamic rendering means the UI changes based on state and API response.

Examples in HireX:

- If `loading` is true, show loader.
- If `error` exists, show error panel.
- If candidates exist, show table.
- If selected candidate exists, show candidate drawer.
- If selected batch changes, show that batch's metrics.

- If validator decision is `REVIEW`, show HR action buttons.

Promise

An API call returns a Promise because the response will come later. We use `await` to wait for that response.

Example:

```
const response = await apiClient.get("/admin/candidates");
```

Meaning:

```
Call backend
  -> wait for response
  -> store response in variable
  -> update UI state
```

Callback

A callback is a function passed or used to run later. In this project:

- `useCallback` creates stable callbacks.
- `.then((response) => ...)` uses callback functions after a Promise completes.
- Button handlers like `onClick={() => openCandidate(...)}` are callbacks triggered by user action.

Async / Await

`async` marks a function that does asynchronous work. `await` pauses inside that function until a Promise finishes.

Frontend example:

```
async function runIntake()
  -> await apiClient.post("/excel-intake/run")
  -> await loadBatches()
  -> await loadDashboard()
```

Backend example:

```
async def run_excel_intake(...)
  -> await ExcelIntakeService(session).run(...)
```

Both frontend and backend are async because API calls and database operations take time.

Parallel API Calls

In `loadDashboard`, we use:

```
Promise.all([
  apiClient.get("/admin/summary"),
  apiClient.get("/admin/candidates")
])
```

Why: summary cards and candidate table are independent, so both can load together. This makes the dashboard faster.

Debounced Loading

The dashboard waits 250 ms before loading:

```
window.setTimeout(loadDashboard, 250)
```

Why: when HR types search text or switches filters, we avoid calling the API too many times immediately.

Polling / Recovery

If the validator request is interrupted, the UI calls:

```
GET /excel-intake/batches/latest
GET /excel-intake/batches/{batch_id}
```

It keeps checking while status is `RUNNING` or `QUEUED`.

Why: the browser request may fail, but backend processing might still have created a batch. Recovery prevents duplicate reruns and helps HR see the real result.

Backend Concepts Used

API

API means Application Programming Interface. In HireX, it is the contract between frontend and backend.

Example:

```
Frontend asks: GET /api/admin/candidates
Backend returns: candidate rows
```

Endpoint

An endpoint is one URL plus one HTTP method.

Example:

```
GET /api/admin/summary
POST /api/excel-intake/run
DELETE /api/excel-intake/batches/{batch_id}
```

Router

A router groups related endpoints.

Example:

```
hirex/api/admin/routes.py
```

contains all admin dashboard endpoints.

Path Parameter

A path parameter is part of the URL.

Example:

```
/admin/candidates/{application_id}
```

Here `application_id` is passed in the URL.

Query Parameter

A query parameter is added after `?`.

Example:

```
/admin/candidates?batch_id=123&workflow_state=PENDING
```

We use query params for filters, search, pagination, batch selection, and selected validator result.

Request Body / Payload

Request body is JSON sent with POST, PUT, or PATCH.

Example:

```
{
  "requirement_id": "REQ-AI-ENGINEER-001",
  "candidate_pool": "synthetic",
  "max_candidates": 1000
}
```

This body is sent to `/api/excel-intake/run`.

Response Model

FastAPI response models are Pydantic schemas that define what the API returns.

Example:

```
response_model=ExcelIntakeResult
```

Why: it keeps frontend and backend response structure predictable.

Dependency Injection

FastAPI injects common dependencies into route functions.

Examples:

```
current_user: User = Depends(get_current_user)
session: AsyncSession = Depends(get_session)
```

Meaning:

- `get_current_user` reads the token and loads the user.
- `get_session` gives a database session.

Service Layer

Routes do not contain heavy business logic. They call services.

Example:

```
POST /excel-intake/run
  -> ExcelIntakeService.run()
```

Why: service layer keeps code clean, testable, and reusable.

Repository Layer

Repositories handle database operations for specific areas.

Example:

```
ValidatorService
  -> ValidatorRepository
  -> database tables
```

ORM

ORM means Object Relational Mapper. HireX uses SQLAlchemy ORM to work with database tables as Python classes.

Example:

```
ValidatorResult table
  -> ValidatorResult Python model
```

Migration

Migration is a versioned database schema change. HireX uses Alembic migrations from `0001` to `0016`.

Why: when we add tables like candidate identity, batch membership, or freshness, migrations keep the database aligned with code.

CORS

CORS allows frontend and backend running on different ports to talk.

In local development:

```
Frontend: http://127.0.0.1:3000
Backend:  http://127.0.0.1:8000
```

`hirex/main.py` allows the frontend origin.

JWT / Bearer Token

JWT token proves the user is logged in.

Flow:

```
POST /auth/login
  -> returns access_token
  -> frontend stores it
  -> Axios sends Authorization: Bearer token
  -> backend validates token
```

HTTP Status Codes

Common statuses in HireX:

Code	Meaning	Example
200	Success	GET candidate list
201	Created	Create review action or run direct validation
204	Deleted/no content	Delete job
401	Not logged in	Missing/invalid token
403	Logged in but not allowed	Candidate tries HR endpoint
404	Not found	Batch/result not found
409	Conflict	Batch already running
422	Validation error	Bad validator input

End-to-End Process Explanation for Lead

Login Process

```
User enters email/password
  -> LoginPage calls POST /auth/login
  -> AuthService verifies password
  -> backend returns JWT token
  -> frontend stores token
  -> frontend calls GET /auth/me
  -> role decides where user is sent
```

Dashboard Load Process

```
Operations page opens
  -> GET /excel-intake/configuration
  -> GET /excel-intake/batches
  -> GET /talent-pool/summary
  -> GET /admin/pool-analytics
  -> GET /admin/reason-bank
  -> GET /admin/summary
  -> GET /admin/candidates
```

Run Validator Process

```
HR selects JD and candidate pool
  -> Run Validator button
  -> optional POST /talent-pool/screen
  -> POST /excel-intake/run
  -> backend creates batch
  -> backend validates candidates
  -> backend exports Excel result files
  -> frontend reloads batches and candidates
```

Candidate Review Process

```
HR opens candidate
  -> GET /admin/candidates/{application_id}
  -> backend returns evidence
  -> HR checks scores and resume sections
  -> if REVIEW, HR submits action
  -> POST /admin/candidates/{application_id}/decision
  -> backend updates HRReviewAction and batch membership
```

Batch View Process

```
Batches page opens
  -> GET /excel-intake/batches
  -> GET /excel-intake/configuration
  -> user selects batch
  -> GET /talent-pool/batches/{batch_id}/memberships
  -> selected batch details render
```

Interview Vocabulary Quick Answers

What is API integration in HireX?

API integration is the connection between Next.js frontend and FastAPI backend using Axios. The frontend sends requests like `/admin/candidates`, backend processes them through routers/services/database, and returns JSON that updates the UI.

What is an endpoint?

An endpoint is a specific backend URL and method, for example `POST /api/excel-intake/run`. It performs one defined action.

What is the main endpoint of the validator flow?

For the dashboard MVP, the main endpoint is `POST /api/excel-intake/run`. It starts the full intake and validator process.

Does frontend directly call the validator endpoint?

Mostly no. The dashboard calls Excel intake. Excel intake internally calls `ValidatorService`. Direct validator endpoints still exist for future direct application validation and agents.

What is a callback?

A callback is a function that runs later. In HireX, click handlers, `.then(...)`, and functions passed to React hooks are callbacks.

What is a Promise?

A Promise is the result of an async operation that will complete later. Axios API calls return Promises.

What is async/await?

`async/await` lets us write asynchronous code in a readable order. We use it for API calls in frontend and database/service calls in backend.

What is dynamic loading?

Dynamic loading means the UI loads data from APIs after the page opens or after filters change. Example: when HR selects a batch, candidate counts and table reload dynamically.

What is rendering?

Rendering means React converts state and JSX into visible UI. When state changes after an API response, React re-renders the affected screen.

What is state?

State is frontend memory for current UI values such as selected batch, candidate list, loading flag, search text, and selected candidate.

What is dependency injection?

Dependency injection is FastAPI automatically providing `current_user` and `session` to route functions.

Why do we use services?

Services keep business logic away from route files. Routes accept requests; services execute process logic.

Why do we use Pydantic schemas?

Pydantic schemas validate request/response data and make API contracts clear for frontend and backend.

Why is batch ID important?

Batch ID separates one validator execution from another. Without it, HR could confuse all-time counts with one job's candidate results.

Why is candidate identity important?

Candidate identity prevents duplicate candidate records when the same person appears in multiple sourcing batches.

Why do we have reusable talent pool APIs?

They let HireX reuse rejected or released candidates for future openings instead of losing candidate metadata.

API Questions Lead Can Ask

1. How many endpoints are there? Answer: 41 total; 40 under `/api/*` and 1 `/health`.
2. Where are endpoints registered? Answer: `hirex/main.py` using `app.include_router(...)`.
3. Which frontend file defines the API base URL? Answer: `frontend/src/api/client.ts`.
4. How is JWT added to requests? Answer: Axios request interceptor reads `hirex_token` from local storage and sets `Authorization: Bearer <token>`.
5. What endpoint runs validation from the HR dashboard? Answer: `POST /api/excel-intake/run`.
6. What service handles the validator batch? Answer: `ExcelIntakeService.run()`.
7. What service calculates the score? Answer: `ValidatorService`.
8. Which API loads the dashboard table? Answer: `GET /api/admin/candidates`.
9. Which API loads dashboard cards? Answer: `GET /api/admin/summary`.
10. Which API opens candidate detail? Answer: `GET /api/admin/candidates/{application_id}` with `validator_result_id` query parameter.
11. Which API records HR decision? Answer: `POST /api/admin/candidates/{application_id}/decision`.
12. Which API lists batches? Answer: `GET /api/excel-intake/batches`.
13. Which API deletes a batch? Answer: `DELETE /api/excel-intake/batches/{batch_id}`.
14. Which API handles reusable pool first? Answer: `POST /api/talent-pool/screen`.
15. Which API lists candidates inside one batch? Answer: `GET /api/talent-pool/batches/{batch_id}/memberships`.
16. What is the difference between `/admin/candidates` and `/talent-pool/batches/{batch_id}/memberships`? Answer: `/admin/candidates` gives dashboard candidate rows with score and HR filters. `/talent-pool/batches/{batch_id}/memberships` gives workflow membership inside a selected batch.
17. Why are there query params in candidate list? Answer: For batch filter, workflow filter, search, limit, and offset pagination.
18. Why do we use `Promise.all`? Answer: To load independent dashboard APIs in parallel and improve speed.
19. Why is there timeout in validator run calls? Answer: Validation can process many candidates, so frontend allows a longer request window.
20. Why is recovery logic needed? Answer: The browser request may fail while backend batch still runs, so frontend checks latest batch instead of blindly rerunning.

Final Short Explanation

HireX APIs are organized by domain. The frontend never directly touches the database. It calls Axios endpoints. FastAPI routes validate auth and input, then call services. Services handle business logic like intake, scoring, dedupe, batch tracking, and HR decisions. SQLAlchemy stores the final state in PostgreSQL. React receives JSON responses, updates state, and re-renders the dashboard dynamically.